Modern Tools of Deep Learning.

Samuel Schepp
Technische Hochschule Mittelhessen
Fachbereich MNI
Master-Seminar
samuel.schepp@mni.thm.de

Abstract

This paper introduces common machine learning tools like TensorFlow, PyCharm and PyPlot, which are used to setup and run machine learning tasks with neural networks. A common "Hello World" script will be developed to show the concept and ideas of these tools. It will make use of the "Fashion MNIST dataset" by Zalando, which will be explored in detail.

In addition, there will be shown how to run trainings on a graphics card and what TensorFlow is not doing.

1 MNIST Datasets

To get started with machine learning, there needs to be data that a model can read and produces a result of. In addition, a model needs much more data to train. In the "Hello World" example, which will be developed later, the "Fashion MNIST dataset" will be used, which is created by Zalando, an online fashion store. Zalando is developing solutions, which can detect and classify pictures of cloths. Their current focus is deep learning using TensorFlow.

A MNIST dataset contains four individual sets. The first set is a list of images, which will be used to train on. The second set is a list of class names, that refers to the actual thing, that is shown on the image at the same index. That means, that image 4 might show a jacket, the class name on set 2, index 4 says "jacket". These two datasets can now be used to train a model, which then should be able to recognize the image.

The remaining two datasets are again pictures and class names of cloths. These are used to test the model. The images are new and different ones, to make sure, the model is not over-fitting. Over-fitting means, that the model is only capable of recognizing the given dataset, but is likely to fail on unknown and new pictures.

1.1 Origin

The acronym "MNIST" database stands for "Modified National Institute of Standards and Technology database". It is a large database of handwritten digits, that is used to train neural networks and image processing systems. There are 60,000 training and 10,000 testing images, which are normalized, centered and scaled to 28 by 28 pixel in size. It is easy for convolutional neural networks, to classify these images at a failure rate of under 1 percent[1].

The images are encoded in a simple byte-array based format, so no conventional image application can open them. TensorFlow is able to decode the images and use them as input, since it became a common format.

Figure 1: The original MNIST database with handwritten digits

[1]Source: http://yann.lecun.com/exdb/mnist/

1.2 Examples

Since the MNIST database is popular amongst machine learning scientists, it became a standard format for other content. Zalando uses it, to provide example data to the public and tries out different learning systems.

They created a database which contains clothing and its names (class). Figure 2 shows an extract of the dataset. It has a size of 26 MB for training and 4.3 MB for test samples. There are 10 classes of clothing: T-shirt/top, trouser, pullover, dress, coat, sandal, shirt, sneaker, bag and ankle boot. It is not a coincidence, that the count of classes here is the same as the original set, which contains handwritten digits. A system, that can train the handwriting, can also train and recognize the clothing, without any reimplementation.

Figure 2: Snippet of the Zalando fashion MNIST.

TensorFlow is able to directly load the fashion MNIST, since it is part of the python package.

```
from tensorflow import keras

fashion_mnist = keras.datasets.
    fashion_mnist
(train_images, train_labels), (test_images
    , test_labels) = fashion_mnist.
    load_data()
```

Listing 1: Loading the fashion MNIST in TensorFlow (Python)

The tuples contain TensorFlow-friendly data, that can directly be used in Keras, which is TensorFlow‘s high level API to configure and run neural networks.

2 TensorFlow

TensorFlow is a software platform for numerical computation, with focus on neural networks and deep learning. It is developed by Google’s AI organization and provides frameworks for many scientific domains.

It provides hardware acceleration for graphics cards and its own chips called ”TensorFlow Processing Unit” (TPU).

Most of the time, the Python package is adequate to perform many basic and advanced machine learning tasks.

2.1 Keras

The Phyton package of TensorFlow contains Keras, which is a high level interface to several deep learning back ends, like Theano, Microsoft Cognitive Toolkit and TensorFlow.

It is the default API for TensorFlow, although it is an independent open source project. That means you will be able to run deep learning algorithms, that are developed for TensorFlow, on different frameworks.

2.2 TensorFlow.js

To train and deploy models for the browser or Node.js, there is the TensorFlow.js JavaScript package. It provides the common low-level algebra library and a high level, Keras-like API for building and running neural networks. In addition, it allows users to convert pre-existing TensorFlow models, to run them in the browser or in a Node.js environment.

This allows JavaScript applications to directly integrate machine learning capabilities into their stack and out source computing power to the client devices, instead of the server.

2.2.1 Browser

The framework can be used directly as a script import from the CDN repository ”https://www.jsdelivr.com”,

```
<script src="https://cdn.jsdelivr.net/npm/
    @tensorflow/tfjs@0.14.1/dist/tf.min.js"
    ></script>
```

Listing 2: Loading TensorFlow.js in the browser via CDN

Now, the Keras API is available for usage and a model can be trained or ran on the browser's JavaScript engine. Hardware acceleration is available via WebGL, which provides a run performance, that is only two times slower then the native Python implementation. For large models, training is up to 15 times slower.

2.2.2 Node.js

For Node.js, there is another package available, which is called "tfjs-node". It uses the native C++ bindings of TensorFlow, but provides the same interface as the browser version. This enables developers to interchange TensorFlow code between browser and server, without any rewrites.

To use the package, install it via NPM.

```
npm install @tensorflow/tfjs-node
```

Listing 3: Installing TensorFlow.js for Node.js

The package has access to all of the common hardware acceleration interfaces of the operating system, like NVIDIA CUDA, which allows models to train and run on a graphics card. That means, that is is possible to write JavaScript neural network code, which runs with maximum performance.

2.3 TensorFlow lite

To use models in mobile applications, there is the "TensorFlow Lite" library, which can be integrated in to existing native iOS and Android mobile applications. It is a C++ library, which comes with native API bindings for the respecting development environments.

Models, that are created in the classic TensorFlow python environment have to be converted to a lightweight and flattened model, that can be shipped with the mobile application.

Even complex models, that are used to classify images in 1000 categories are only 17 MB in size.

The key requirement of a mobile application that uses machine learning is speed and battery performance. TensorFlow Lite is optimizing models by making a trade off between model complexity and size. The tools are helping developers to fine tune the model size, by allowing adjustments even while converting the model.

A common image classification model's accuracy H is growing with file size D by a factor of $H \approx \sqrt{D}$. As a consequence, the models file size can be optimized at first, without compromising much performance. A model can also be optimized by avoiding over fitting and training with distinguished and good data. However, these steps require adequate scientific approaches.

2.3.1 Usage example iOS

To use TensorFlow Lite models in an iOS applications, the C++ library has to be built from source and linked against. This is achieved by cloning the repository and running some shell scripts in the directory. A library is built, which is called "libtensorflow-lite.a". The iOS project now has to be configured with the correct library and header search paths.

The API is a low level interface and some boilerplate code is needed to make a simple image recognition demonstration working. To classify an image, the model has to run against a pixel buffer `CVPixelBufferRef` and the result will be written into a vector of class labels and their probability `std::vector<std::pair<float, int>>`. This requires about 100 lines of objective C and C++ code.

For a more simple API, Apple created CoreML, which is yet restricted to common use cases like image and text classification. It supports TensorFlow lite models as base format.

2.4 Limits

Originally, TensorFlow was developed to create an open-source data flow oriented programming library. The goal was to enable symbolic math operations, lazy evaluation and hardware acceleration, which runs platform independent.

A simple multiplication of two constants can be run with the following Python code.

```
sess = tf.Session()
x = tf.constant(3, dtype=tf.int8)
y = tf.constant(2, dtype=tf.int8)
z = tf.multiply(x, y)
sess.run(z)
# 6
```

Listing 4: Multiplying two constants in Tensorflow

The operation are represented in a graph, which will be evaluated, when the session in

ran.

Neural networks are only a part of TensorFlow, which evolved to the most important aspect, since the maintainers are promoting almost exclusively the machine learning facets of TensorFlow.

With this idea behind TensorFlow, it is important to known, what it is *not* doing. It is not designed to support data scientist in collecting, managing, preprocessing and analyzing data and model results. It is not possible, to let TensorFlow make semantic inferences on the inputs or outputs. It only runs operations of maths and information science, which makes it low level on this point of view.

3 PyPlot

To be able to work with data and analyze results, PyPlot offers a Python framework to visualize data directly in the IDE. It can be used in the TensorFlow-scripts to output images, graphs, tables and diagrams. The API is designed to use a MATLAB-like interface and enables a wide variety of customization options.

Listing 5 generates a plot of two arrays, which represent the sinus function, by plotting two arrays, that contain discrete example values. Variable x is an array that is generated by creating values from 0 to 2π with an interval of 0.5, which results in 13 values. y is the result of mapping x using the sinus function. Running the script in an supported IDE like Jetbrains PyCharm, enables interactive display of the generated plots by calling plt.show().

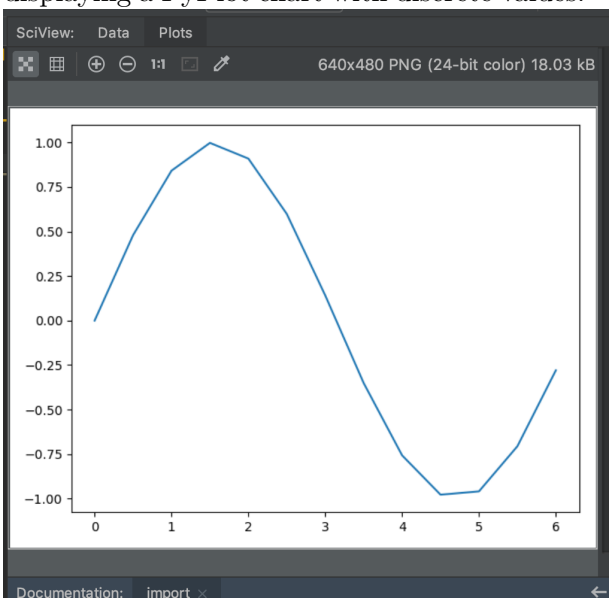
```
import numpy as np
import matplotlib.pyplot as plt

x = np.arange(0, 2*np.pi, 0.5);
y = np.sin(x)
plt.plot(x, y)
plt.show()
```

Listing 5: Generating a plot of sin(x) with some discrete values.)

Figure 3 shows a screenshot of the PyCharm IDE. The window SciView displays all the plots, that are generated while the code is ran.

This helps to understand machine learning algorithms, since it can show input and output data. Image recognition algorithms can output the image first, then the processed one and then the result after the session finished. An examples can be found in section 5, where

Figure 3: Screenshot of Jetbrains PyCharm displaying a PyPlot chart with discrete values.



there will the results be analyzed using PyPlot. Figure 2 is also generated using PyPlot.

4 Hardware Acceleration

When using Tensorflow's high-level APIs like Keras, most of the optimizations are already included. There are some ways, a developer can still improve performance.

4.1 CPU

When using CPU specific improvements like Intel MKL-DNN[2] or targeting special architectures like Xeon Phi, TensorFlow should be built from source, so that all of the supported instructions are available.

Since different kinds of neural networks require other optimizations, a general performance guide is not possible. For Example: When training CNN[3] based models, TensorFlow should be built with MKL, so that AVX[4] is available. That way, modern consumer processors from Intel are high-performing and can train CNN models up to 9 times faster[5]. There are many configuration options available, which can improve performance for spe-

[2] Intel Math Kernel Library for Deep Neural Networks (Intel MKL-DNN)

[3] Convolutional Neural Networks

[4] Advanced Vector Extensions

[5] https://towardsdatascience.com/stop-installing-tensorflow-using-pip-for-performance-sake-5854f9d9eb0c

cial setups even further.

4.2 GPU

Using a GPU can also increase performance drastically. NVIDIA CUDA[6] is a parallel computing platform and programming interface to abstract parallelism, which lets users run single threaded tasks on the CPU and multi-threaded task on the GPU. TensorFlow takes advantage of CUDA and optimizes training sessions to run multi-threaded. Other libraries like NumPy are also optimized , so that pre- or post-processing can be sped up too. In some situations, a consumer grade graphics card can be up to 15 times faster[7] than a consumer grade desktop processor.

At the moment, CUDA is the only hardware platform, that is supported officially in TensorFlow's GPU package. It is possible to use external GPUs via macOS's eGPU interface.

4.3 Device selection

If enabled, TensorFlow will run all of it's computations on the GPU, otherwise it will fallback to the CPU. Running something on selected devices can be accomplished by using a 'with' block.

```
with tf.device('/cpu:0'):
  a = tf.constant([1.0, 2.0, 3.0, 4.0,
    5.0, 6.0], shape=[2, 3], name='a')
  b = tf.constant([1.0, 2.0, 3.0, 4.0,
    5.0, 6.0], shape=[3, 2], name='b')
c = tf.matmul(a, b)
```

Listing 6: Running the instantiation on the CPU, the multiplication on the GPU.

In listing 6, the allocation and configuration of the matrix and assignment to a and b are done on the CPU. In Python, the 'with' keyword provides an object to a block. It is used here to define the device for the statements.

The actual matrix multiplication is done on the GPU, because the statement is outside the 'with' block and the GPU it is the default device.

5 Hello World

Now, a simple example application can combine all of the previously described tools. This example will use TensorFlow to train classification of images from the Zalando fashion MNIST. See Figure 2 for a demo snippet of the data set.

To get started, a modern Python development environment has to be set up. Using pip, the TensorFlow Python package can be installed, which contains many libraries that will handle most of the common machine learning tasks.

```
pip install tensorflow
```

Listing 7: Install TesorFlow Python packages

Next, a single Python file is needed to setup a simple neural network. The next section will cover all of the important code snippets, that are needed to implement an example application of a neural model. The neural network will classify images of clothes as described in section 1.

First of all, the recently installed Python libraries have to be imported into the source file. This is done by a standard import statements in listing 8.

```
import tensorflow as tf
from tensorflow import keras
import numpy as np
```

Listing 8: Importing TensorFlow and it's base network API Keras.

Now the MNIST data can be set up by using a handy Keras interface to load the prebundled fashion mnist.

```
(train_images, train_labels), (test_images
    , test_labels) = keras.datasets.
    fashion_mnist.load_data()
class_names = ['T-shirt/top', 'Trouser', '
    Pullover', 'Dress', 'Coat',
                'Sandal', 'Shirt', 'Sneaker
    ', 'Bag', 'Ankle boot']

train_images = train_images / 255.0
test_images = test_images / 255.0
```

Listing 9: Setup of the training and test datasets.

The statement *keras.datasets.fashion_mnist.load_data()* returns a tuple of training data and test data, of which each is a tuple of input and output. Since the images are grayscale images with a color value from 0 to 255, all values are divided by 255, to normalize the value scale to 0 to 1. This is needed to make sure that the values can correctly be used inside the neural network. It is required, that a full value is 1, because if it is multiplied by 1, it must still yield the value 1.

[6] Compute Unified Device Architecture

[7] https://medium.com/@andriylazorenko/tensorflow-performance-test-cpu-vs-gpu-79fcd39170c

Now it is time to setup the model. The model, that is setup in listing 10 has 784 input nodes, 60 middle layer nodes and 10 output nodes.

```
model = keras.Sequential([
    keras.layers.Flatten(input_shape=(28,
    28)),
    keras.layers.Dense(60, activation=tf.
    nn.relu),
    keras.layers.Dense(10, activation=tf.
    nn.softmax)
])
model.compile(optimizer=tf.train.
    AdamOptimizer(),
              loss='
    sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

Listing 10: Setup of the model.

Because the images have a size of 28 by 28 pixels, there are 784 in total. The model is a fully connected neural network, where every node of one layer is connected to every node of the following layers. There are three layers in total.

keras.layers.Dense declares, that the nodes are fully connected to their parents.

The network is configured to use the Adam algorithm for optimization. It is a common algorithm, that is used a lot in deep learning and defines a specific way of updating the network weights while training. The network weights are the core elements of decision making.

tf.nn.relu represents the Rectified Linear Unit activation function. An activation function is applied to the outputs of a node. The relu function will return 0 if the input was negative. Otherwise, it returns the input. It is used to secure the neural network from unwanted values.

tf.nn.softmax makes sure, that all output values will be scaled to values between 0 and 1. It is needed to make sure, that the outputs are valid probability values.

A loss function determines, how far the result value deviates from the correct value.

The next task is to let the model train it's weights. To do so, a fit task can be run on the model. Since the data already is in the correct format, a simple call will train the model.

```
model.fit(train_images, train_labels,
    epochs=10)
```

Listing 11: Training the model by using the train value pairs.

Listing 11 trains the model 10 times, by using all the images of the train set. The model can now be exported as a TensorFlow light model, to be used in mobile applications or on the server. But the interesting part is the testing, using the provided test pair.

To perform a model test, the code of listing 12 can be used.

```
predictions = model.predict(test_images)
```

Listing 12: Testing the model by using the given test set.

The variable predictions now contains an array of ten tuples of numbers. The numbers of the ten tuple represent the probabilities of all the 10 classes (T-shirt, pullover, dress, ...). That means, for every item, a probability map is returned.

5.1 Evaluation

Now, the results have to be evaluated. To count the success and failure rate, a simple algorithm needs to iterate over all result tuples and compare the class with the highest prediction value with the given correct index. This can be done by using the code example of listing 13.

```
missed = 0
success = 0
for i in range(predictions.shape[0]):
    predicted_label = np.argmax(
    predictions[i])
    true_label = test_labels[i]
    if predicted_label != true_label:
        missed += 1
    else:
        success += 1
```

Listing 13: Akkumulate succes and failure count.

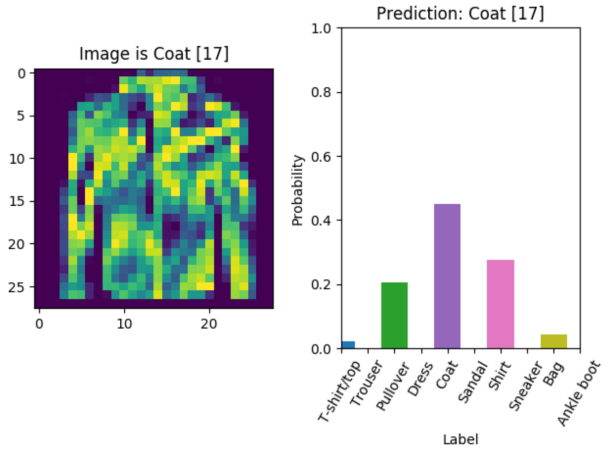
np.argmax returns the index of the highest value in an array. This index directly represents the class label. The label is then compared to the given reference label. If they match, the success variable is incremented. Otherwise, the missed variable is incremented.

Figure 4 represents an example prediction tuple, which is directly extracted from the ten output nodes of the neural network.

5.2 Performance

There are many ways to optimize a model, into being more accurate or faster in training times. The two first parameters that can be changed are the epoch count and the node count of the

Figure 4: Graph, representing the prediction tuple of one test example.



middle layer. An epoch is defined as *one pass over the entire dataset*[8].

Because of a neural networks nature, an increase of the epoch count does not necessarily result in an increase of accuracy. This is related to over fitting, which lets a model that has been trained with to many epochs my only classify it's training data correctly. It is important to find a good fit of epochs and node counts. This can be achieved primarily with high quality training data and *trial and error*.

Figure 5 displays the impact of changing the epoch and nodes count to the accuracy and the training time.

Figure 5: Training time and test accuracy overview.

Accuracy [%]		Nodes					
		10	30	100	500	1000	5000
Epochs	1	79,00%	81,00%	82,30%	80,90%	81,60%	82,20%
	5	81,10%	84,00%	85,90%	85,00%	86,40%	86,20%
	20	83,00%	85,40%	87,00%	88,30%	88,10%	88,10%
	100	82,30%	85,00%	87,50%	88,00%	88,20%	87,80%

Time [Seconds]		Nodes					
		10	30	100	500	1000	5000
Epochs	1	1,162	1,278	1,667	2,722	5,035	25,189
	5	4,784	5,474	6,710	12,052	23,346	126,669
	20	18,270	20,735	25,980	45,891	92,223	493,791
	100	92,680	102,581	128,875	231,419	461,358	2456,508

Generally, higher node and epoch count does result in a higher accuracy, even though the coherence has some exceptions. Those exceptions are results, that are worse, even though the parameters are higher. This can be related to the low amount of training or test data, which causes some random tests to fail, which has a big impact on values.

The best combination on this experiment were 500 nodes and 20 epochs, which resulted in an accuracy of 88,3% and a training duration of 45,891 seconds. The worst combination is the one with 5000 nodes and an epoch count of 100. This run resulted in an high accuracy of 87,8%, but has an unsatisfying training time of 40 minutes.

The epoch count generally increases the training time by a linear factor, while the node count's impact cannot be determined, because of parallel optimization mechanisms. But high node counts (starting from 500) begin to be a linear factor as well.

The tests were run on an iMac, running on macOS 10.14.5 on a 4.2 GHz Intel Core i7.

5.3 Hardware Acceleration

GPU acceleration as described in section 4.2 is supported on Windows, while other platform support is provided by community projects as experiments. This section's examples are run on a Windows computer that is equipped with an 3.6 GHz Intel Core i7-9700K CPU and an NVIDIA GeForce RTX 2080 graphics card. This card is optimized for computer games, but also has CUDA cores, used for deep learning acceleration. Running the same example, that is described in the previous section on a GPU, the training performance is two times higher, compared to the CPU on the same system. See figure 6 and 7 for a comparison.

Figure 6: GPU and CPU performance using 500 nodes and 20 epochs.

	Test Accuracy	Training Times
CPU	87.3 %	107 seconds
GPU	88.1 %	49.25 seconds

Figure 7: GPU and CPU performance using 5000 nodes and 50 epochs.

	Test Accuracy	Training Times
CPU	88.2 %	1341 seconds
GPU	87.5 %	171 seconds

[8] `https://keras.io/getting-started/faq/\#what-does-sample-batch-epoch-mean`

Figure 6 indicates an performance improvement of 54 % and figure 7 an improvement of 87 %. Further test scenarios show, that especially high node counts benefit from graphics card support, while the epoch count still has an linear impact on the training times.

It is clear, that the dedicated hardware support and acceleration described in 4.2 are working as expected. However, the graphics card on the windows machine was slower then the CPU in the Mac that was used in figure 5. This behavior may be due to the different host platforms which may use different optimization methods.

6 Conclusion

Deep learning has long since arrived in the broad development masses and can be utilized by everyone in common development environments. An easy setup process and a lot of resources that are supported and created by big players like Google, Amazon, Facebook and Microsoft ensure that neural network approaches can be tried out quickly. The TensorFlow platform is by far the most relevant development tool and can be integrated into many systems like client/server application of even mobile and web apps.

While the first getting started projects are easy and fun to play around with, more advanced topics and use-cases require comprehensive knowledge of mathematics and statistics. TensorFlow hardware acceleration is only supported on one hardware platform (CUDA), which uses proprietary hardware and driver components. Decisions or results of neural networks are unpredictable and it is not clear, how characteristics of the input influence the outcome. This makes it hard to prove the correctness of a model, because new input always results in an wayward result.

There is an inconsistency that occurred while evaluating the test result that are displayed in figure 5, 6 and 7. The results suggest, that the run using 500 nodes and 20 epochs has been faster on a 4,2 GHz Intel quad core processor than on an Nvidia Gforce RTX 2080 graphics card using CUDA. The system that was used for running the neural network on a processor was an Apple iMac from 2017, running macOS. The run on the graphics card was performed on a 2018 Windows PC running gaming hardware. One explanation for the performance loss can be the different platforms. It is possible, that running on CUDA creates an overhead, that was to big for the small neural network, so that the conduction work was fast and direct on the CPU, but slow and costly on the GPU, because meta data has to be transferred between CPU and GPU. The high frequency of the CPU presumably resulted in a fast training.

Testing this theory with a higher node and epoch count was not successful as the wait times increased massively and the same results showed up. Different kinds of neural networks with more inner layers and algorithms could be used, but the complexity was too high, and the test accuracy dropped significantly.

References

Fashion-MNIST

`https://github.com/zalandoresearch/fashion-mnist`

Accessed: 2019-01-12.

Zalando Research

`https://research.zalando.com`

Accessed: 2019-01-12.

MNIST Database

`http://yann.lecun.com/exdb/mnist/`

Accessed: 2019-01-12.

TensorFlow Processing Unit

`https://cloud.google.com/tpu/?hl=de`

Accessed: 2019-01-12.

TensorFlow Lite

`https://www.tensorflow.org/lite/performance/best_practices`

Accessed: 2019-01-15.

Mobilenet_v2 Model

`https://github.com/tensorflow/models/blob/master/research/slim/nets/mobilenet_v1.md`

Accessed: 2019-01-15.

TensorFlow performance test: CPU VS GPU

`https://medium.com/@andriylazorenko/tensorflow-performance-test-cpu-vs-gpu-79fcd39170c`

Accessed: 2019-03-06.

Stop Installing Tensorflow using pip for performance sake!

https://towardsdatascience.com/stop-installing-tensorflow-using-pip-for-performance-sake-5854f9d9eb0c

Accessed: 2019-03-06.

Training Your Neural Net with eGPU Acceleration on Mac with Tensorflow 1.5

https://medium.com/@jianshi_94445/training-your-neural-net-with-egpu-acceleration-on-mac-with-tensorflow-1-5-b2b729f4e408

Accessed: 2019-03-06.

5 Trends Emerge in the Gartner Hype Cycle for Emerging Technologies, 2018

https://www.gartner.com/smarterwithgartner/5-trends-emerge-in-gartner-hype-cycle-for-emerging-technologies-2018/

Accessed: 2019-05-18.